Белорусский государственный университет

Факультет прикладной математики и информатики

Лабораторная работа №1

Цветовые модели: CMYK, RGB, HSV

Выполнил:

студент 3 курса

группы 4

Филюта Артём Витальевич

Проверил: Соловей Сергей Сергеевич

Минск, 2026

Содержание

1 Введение **2**

2 Стек технологий **2**

3 Математические основы преобразований **2**

3.1 RGB → CMYK 3

3.2 CMYK → RGB 3

3.3 RGB → HSV 3

3.4 HSV → RGB 3

4 Архитектура приложения **4**

4.1 Хранение состояния 4

4.2 Необратимые точки и память оттенка и насыщенности 4

4.3 Цикл обновления 4

5 Пользовательский интерфейс **5**

5.1 Отрисовка палитры 6

6 Возникшие проблемы и их решения **6**

6.1 Необратимость преобразований 6

6.2 Смещение маркера на палитре при чёрном цвете 6

6.3 Согласование отображения после пересчёта 6

7 Выводы **7**

1. Введение

Цель работы: изучить цветовые модели RGB, CMYK и HSV, реализовать взаимные преобразования между ними, а также разработать интерактивное приложение, позволяющее пользователю задавать цвет тремя различными способами в каждой из трёх моделей с автоматическим пересчётом компонент во всех остальных представлениях.

Задачи:

1. Разобрать математические формулы преобразований RGB $\leftrightarrow$ CMYK и RGB $\leftrightarrow$ HSV, а также преобразований между CMYK и HSV через RGB.
2. Разработать архитектуру приложения, обеспечивающую согласованное хранение состояния цвета.
3. Реализовать интерфейс с тремя способами задания цвета: палитра (клик и перетаскивание), ползунки, поля точного ввода.
4. Обеспечить автоматический пересчёт цвета во всех моделях при изменении любой из компонент.

2. Стек технологий

Для реализации приложения использовались следующие технологии:

- **Python 3** — вся логика приложения: преобразования цветовых моделей, обработка событий пользователя, синхронизация состояния.
- **tkinter / ttk** — стандартная библиотека для построения графического интерфейса: окна, рамки, ползунки `ttk.Scale`, поля ввода `ttk.Entry`, холсты `Canvas` для палитры.
- **tkinter.colorchooser** — системный диалог выбора цвета.

Приложение работает полностью локально и не использует внешних библиотек. Формулы преобразования вынесены в отдельный модуль `colormodels.py` и реализованы вручную, без готовых функций конвертации цвета.

3. Математические основы преобразований

Значения R, G, B нормируются к диапазону $[0, 1]$ делением на 255. Далее $\max = \max(R, G, B)$, $\min = \min(R, G, B)$, $\Delta = \max - \min$.

3.1. RGB → CMYK

$$K = 1 - \max(R, G, B) \tag{1}$$

$$C = \frac{1 - R - K}{1 - K}, \qquad M = \frac{1 - G - K}{1 - K}, \qquad Y = \frac{1 - B - K}{1 - K} \tag{2}$$

При $K = 1$ (чистый чёрный цвет) компоненты C, M, Y полагаются нулевыми, поскольку восстановить их из чёрного невозможно.

3.2. CMYK → RGB

$$R = (1 - C)(1 - K), \qquad G = (1 - M)(1 - K), \qquad B = (1 - Y)(1 - K) \tag{3}$$

3.3. RGB → HSV

$$V = \max, \qquad S = \begin{cases} 0, & \max = 0, \\ \dfrac{\Delta}{\max}, & \max > 0. \end{cases} \tag{4}$$

Оттенок H вычисляется по доминирующему каналу:

$$H = \begin{cases} 0, & \Delta = 0, \\ 60^\circ \cdot \left(\dfrac{G - B}{\Delta} \bmod 6 \right), & \max = R, \\ 60^\circ \cdot \left(\dfrac{B - R}{\Delta} + 2 \right), & \max = G, \\ 60^\circ \cdot \left(\dfrac{R - G}{\Delta} + 4 \right), & \max = B. \end{cases} \tag{5}$$

3.4. HSV → RGB

Промежуточные величины:

$$C = V \cdot S, \qquad X = C \cdot \left(1 - \left| \frac{H}{60^\circ} \bmod 2 - 1 \right| \right), \qquad m = V - C. \tag{6}$$

Далее в зависимости от сектора H формируется промежуточная тройка (R', G', B'):

H	(R', G', B')	H	(R', G', B')
$[0^\circ; 60^\circ)$	$(C, X, 0)$	$[180^\circ; 240^\circ)$	$(0, X, C)$
$[60^\circ; 120^\circ)$	$(X, C, 0)$	$[240^\circ; 300^\circ)$	$(X, 0, C)$
$[120^\circ; 180^\circ)$	$(0, C, X)$	$[300^\circ; 360^\circ)$	$(C, 0, X)$

и итоговые значения получаются как $R = R' + m$, $G = G' + m$, $B = B' + m$ (после чего умножаются на 255).

4. Архитектура приложения

Приложение состоит из двух файлов:

- `colormodels.py` — четыре функции преобразования (`rgb_to_cmyk`, `cmyk_to_rgb`, `rgb_to_hsv`, `hsv_to_rgb`) и вспомогательные функции для HEX;
- `main.py` — класс `App` с интерфейсом и логикой обновления состояния.

4.1. Хранение состояния

Единственным источником истины является текущий цвет в RGB, хранящийся в виде трёх вещественных чисел в диапазоне $[0; 255]$ (атрибут `self.rgb`). Вещественные значения (а не целые) нужны для того, чтобы при цепочке преобразований не накапливались ошибки округления: поля RGB округляются только при отображении.

Когда пользователь изменяет цвет в любой из моделей, значения переводятся в RGB, после чего все остальные представления пересчитываются из него:

```
def set_from(self, model, vals, source, kind):
    if model == "HSV":
        self.hue = vals[0]
        self.sat = vals[1]
    r, g, b = TO_RGB[model](*vals)
    self.rgb = [cm.clamp(r, 0, 255), cm.clamp(g, 0, 255),
                cm.clamp(b, 0, 255)]
    self.refresh(source, kind)
```

4.2. Необратимые точки и память оттенка и насыщенности

Хранение только RGB приводит к потере информации в особых точках:

- у серых цветов ($\Delta = 0$) оттенок H не определён, при пересчёте RGB $\to$ HSV он обнулялся бы, и ползунок оттенка «отскакивал» в ноль;
- у чёрного цвета ($V = 0$) не определены ни H, ни S.

Поэтому приложение дополнительно хранит последние осмысленные значения оттенка `self.hue` и насыщенности `self.sat` и подставляет их в вырожденных случаях.

4.3. Цикл обновления

Метод `refresh` вычисляет значения во всех трёх моделях и раскладывает их по виджетам:

```
def refresh(self, source, kind):
    r, g, b = self.rgb
```

```
    hsv = list(cm.rgb_to_hsv(r, g, b))
    if hsv[1] < EPS or hsv[2] < EPS:
        hsv[0] = self.hue
    else:
        self.hue = hsv[0]
    if hsv[2] < EPS:
        hsv[1] = self.sat
    else:
        self.sat = hsv[1]
    values = {"RGB": [r, g, b],
              "CMYK": list(cm.rgb_to_cmyk(r, g, b)),
              "HSV": hsv}
    self._lock = True
    try:
        for model, vals in values.items():
            for i, v in enumerate(vals):
                skip_scale = (model == source and kind == "scale")
                skip_entry = (model == source and kind == "entry")
                if not skip_scale:
                    self.scales[(model, i)].set(v)
                if not skip_entry:
                    self.vars[(model, i)].set(fmt(model, v))
        ...
    finally:
        self._lock = False
```

Два механизма защищают интерфейс от нежелательных эффектов. Во-первых, виджет, которым пользователь управляет в данный момент (ползунок или поле ввода), не перезаписывается, иначе во время набора числа текст «прыгал» бы. Во-вторых, флаг `_lock` блокирует обработчики событий на время обновления: без него программная установка значения ползунка вызывала бы его обработчик и приводила бы к бесконечной рекурсии пересчётов.

5. Пользовательский интерфейс

Интерфейс приложения включает:

- **область предпросмотра** — образец текущего цвета, поле HEX и кнопка вызова системного диалога выбора цвета;
- **палитру**: квадрат насыщенности и яркости и горизонтальную полосу оттенков, поддерживающую клик и перетаскивание;
- **три панели моделей** (RGB, CMYK, HSV), каждая из которых содержит ползунок и поле точного числового ввода для каждой компоненты.

Таким образом, цвет можно задать тремя независимыми способами: палитрой, ползунком

и текстовым полем. Значения вне допустимого диапазона обрезаются, а некорректный текст игнорируется, после чего в поле возвращается последнее корректное значение.

5.1. Отрисовка палитры

Квадрат насыщенности и яркости рисуется в объект `tk.PhotoImage` построчной заливкой: по оси X откладывается насыщенность S от 0 до 100%, по оси Y — яркость V от 100% до 0 (сверху вниз) при фиксированном оттенке H. Цвет каждого пикселя вычисляется функцией `hsv_to_rgb`. Полоса оттенков строится один раз при запуске. Квадрат перерисовывается только тогда, когда оттенок изменился более чем на $0{,}5°$, что убирает лишние затраты при движении ползунков S и V.

Положение маркера определяется формулами

$$x = \frac{S}{100}(w-1), \qquad y = \left(1 - \frac{V}{100}\right)(h-1), \tag{7}$$

где w и h — размеры квадрата в пикселях.

6. Возникшие проблемы и их решения

6.1. Необратимость преобразований

Как отмечалось выше, при хранении одного лишь RGB теряется информация, которую пользователь задал в исходной модели: у серого цвета обнуляется оттенок, у чёрного — оттенок и насыщенность. Решение — хранение последних осмысленных значений `hue` и `sat`, которые подставляются при пересчёте в вырожденных точках.

6.2. Смещение маркера на палитре при чёрном цвете

При вводе значения 100 в поле K (или при установке $C = M = Y = 100\%$, $R = G = B = 0$) маркер на палитре резко смещался к левому краю. Причина — у чёрного цвета $V = 0$, и формула RGB $\to$ HSV возвращает $S = 0$ (по определению: $S = 0$ при $\max = 0$), а координата маркера по оси X пропорциональна S. Решение: при $V = 0$ значение насыщенности берётся из памяти `self.sat`, и маркер остаётся на нижнем краю палитры на той же горизонтали, где был до этого.

6.3. Согласование отображения после пересчёта

Потребовалось добиться, чтобы поле, в которое пользователь вводит число, не перезаписывалось во время набора, а ползунок, который тянет пользователь, не получал собственные значения обратно (иначе он «дрожит» из-за округления). Это решено параметрами `source` и `kind` метода `refresh`, описанными в разделе об архитектуре.

7. Выводы

В ходе лабораторной работы были изучены три цветовые модели — RGB, CMYK и HSV — и реализованы взаимные преобразования между ними на языке Python. Модели отражают разные аспекты восприятия и воспроизведения цвета: RGB описывает аддитивное смешение излучений (экраны), CMYK — субтрактивное смешение красок (печать), а HSV является перцептивной переупаковкой RGB, удобной для интерактивного выбора цвета.

Практическая часть показала, что основная сложность заключается не столько в математике преобразований, сколько в хранении состояния: при переходе через RGB в особых точках (серые цвета, чистый чёрный) теряется часть информации, заданной пользователем. Хранение вещественных значений RGB и запоминание последних значений оттенка и насыщенности позволило:

- корректно обрабатывать крайние случаи ($K = 100\%$, $V = 0$, серые цвета);
- устранить «прыжки» маркера на палитре и «отскакивание» полей ввода;
- обеспечить согласованное обновление всех представлений цвета при любом способе ввода.

Приложение реализовано на стандартной библиотеке Python (tkinter) без внешних зависимостей.